

CineMax

Manuale Tecnico

Sistema di Gestione Cinema Monosala

Maggio 2026 · A.A. 2025/2026

Laboratorio Interdisciplinare A

Università degli Studi dell'Insubria · Dipartimento di Informatica a Como

Indice

1 Introduzione

2 Installazione

2.1 Requisiti di sistema

2.2 Setup ambiente sviluppo

2.3 Compilazione e build

3 Architettura e struttura del codice

3.1 Architettura a layer

3.2 Struttura package

3.3 Classi principali

3.4 Strutture dati

3.5 Algoritmi principali

3.6 Persistenza e file

4 Dataset di test

5 Limiti della soluzione

6 Sitografia e bibliografia

1. Introduzione

Questo documento descrive dal punto di vista tecnico il progetto CineMax sviluppato per il Laboratorio Interdisciplinare A dell'Università degli Studi dell'Insubria.

L'obiettivo del progetto era realizzare un sistema completo per la gestione di un cinema monosala da 200 posti utilizzando Java 21 e una struttura organizzata in package separati.

L'applicazione permette:

- gestione film e proiezioni;
- autenticazione utenti;
- registrazione clienti;
- gestione prenotazioni;
- controllo disponibilità posti;
- ricerca proiezioni tramite filtri.

Il programma utilizza una TUI (Text User Interface), quindi tutta l'interazione avviene tramite menu numerici da terminale. I dati vengono salvati tramite file CSV e TXT.

2. Installazione

2.1 Requisiti di sistema

Componente	Requisito	Note
Sistema operativo	Windows/Linux/macOS	Qualsiasi sistema con JVM
Java JDK	Versione 21	Utile per compilare ed eseguire il progetto
RAM	512 MB minimi	1 GB consigliato
Spazio su disco	~50 MB	Per progetto e dataset
Terminale	cmd / bash / PowerShell	Per avviare il programma
Connessione internet	Non necessaria	Serve solo per il download iniziale

2.2 Setup ambiente sviluppo

Per lavorare sul progetto è consigliato utilizzare IntelliJ IDEA.

L'IDE permette di compilare velocemente il progetto e generare automaticamente la JavaDoc.

Per verificare l'installazione di Java:

```
java -version  
javac -version
```

Se Java non è installato correttamente il terminale restituirà un errore.

2.3 Compilazione e build

Il progetto può essere compilato direttamente da terminale oppure tramite IDE.

Comandi principali:

```
javac -d bin/classes src/**/*.java
```

Per generare il JAR:

```
jar cfm CineMax.jar MANIFEST.MF -C bin/classes .
```

Per eseguire il programma:

```
java -jar CineMax.jar
```

3. Architettura e struttura del codice

3.1 Architettura a layer

Il progetto è organizzato in più layer separati:

- UI → gestione menu e interazione utente;
- Service → logica applicativa;
- Repository → lettura e scrittura file;
- Model → rappresentazione oggetti del dominio;
- Util → classi di supporto.

Questa suddivisione permette di separare la logica del programma dalla gestione dei dati e dall'interfaccia.

3.2 Struttura package

Nel progetto sono presenti package dedicati alle diverse responsabilità:

- cinemax.model
- cinemax.service
- cinemax.repository
- cinemax.ui
- cinemax.util

Ogni package contiene classi specifiche collegate ad una determinata parte dell'applicazione.

3.3 Classi principali

Film

Contiene le informazioni principali di un film.

Proiezione

Gestisce data, orario e costo delle proiezioni.

Prenotazione

Rappresenta una prenotazione effettuata da un cliente.

Utente

Classe base per la gestione utenti.

AuthService

Gestisce login e autenticazione.

ProiezioneRepository

Gestisce salvataggio e caricamento proiezioni.

PrenotazioneService

Gestisce controlli sui posti disponibili.

3.4 Strutture dati

La struttura dati principale utilizzata nel progetto è ArrayList.

Le liste vengono caricate completamente in memoria all'avvio e vengono usate per:

- proiezioni;
- utenti;
- prenotazioni;
- film.

La scelta di ArrayList è stata fatta perché semplice da utilizzare e sufficiente per le dimensioni del dataset previste dal progetto.

3.5 Algoritmi principali

Ricerca proiezioni:

la ricerca scorre tutte le proiezioni presenti in memoria e applica più filtri contemporaneamente.

Controllo sovrapposizioni:
prima di inserire una nuova proiezione il sistema verifica
che non esistano sovrapposizioni di orario.

Calcolo posti disponibili:
i posti disponibili vengono calcolati sottraendo i biglietti prenotati
dal totale della sala.

Generazione codice prenotazione:
il codice viene creato automaticamente nel formato:
CMX-YYYY-NNN

3.6 Persistenza e file

Tutti i dati vengono salvati nella cartella data/ tramite file CSV e TXT.

proiezioni.csv:
contiene le proiezioni separate da virgola.

utenti.txt:
contiene utenti e password cifrate tramite hash MD5.

prenotazioni.txt:
contiene le prenotazioni effettuate dai clienti.

Il parsing dei file viene effettuato tramite `String.split()`.

4. Dataset di test

Il progetto contiene un dataset iniziale utilizzato per testare rapidamente
tutte le funzionalità del sistema.

Sono presenti:

- proiezioni già caricate;
- utenti precaricati;
- prenotazioni di esempio.

I file si trovano nella cartella data/.

5. Limiti della soluzione

Limitazione	Spiegazione
Solo interfaccia testuale	Nessuna GUI grafica prevista dal progetto.
Nessun database	I dati vengono salvati solo su file CSV e TXT.
Nessun accesso concorrente	Il sistema non gestisce utenti simultanei.
Nessun logging	Gli errori vengono mostrati solamente a terminale.

6. Sitografia e bibliografia

- Oracle Java Documentation
- GitHub
- Git SCM
- JetBrains IntelliJ IDEA
- Materiale del corso Laboratorio Interdisciplinare A